

中山大學
SUN YAT-SEN UNIVERSITY

# Lecture 19
# Tree Extensions

## Data Structures and Algorithms

Zizhen Zhang, School of Computer Science and Engineering, Sun Yat-sen University

zhangzizhen@gmail.com

# Outline

- Huffman Trees
- Threaded Binary Trees
- Lexicographic Search Trees: Tries*
- Splay Trees*



# Huffman Coding 哈夫曼编码

- Consider the following data, which is taken from a Postscript file.

| Char | Freq |
|---|---|
| 5 | 1294 |
| 9 | 1525 |
| 6 | 2260 |
| 4 | 2561 |
| 2 | 4442 |
| 3 | 5960 |
| 7 | 6878 |
| 8 | 8865 |
| 1 | 11610 |
| 0 | 70784 |

- Notice that there are many more occurrences of 0 and 1 than the other characters.

# Huffman Coding

- What would happen if we used the following code to store the data rather than the fixed length code?

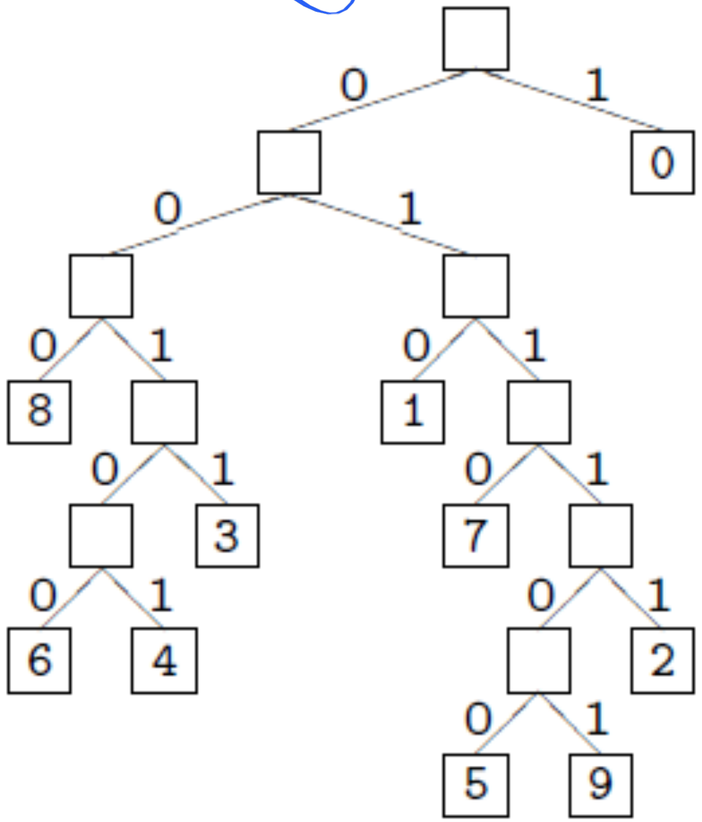
| Char | Code |
|---|---|
| 0 | 1 |
| 1 | 010 |
| 2 | 01111 |
| 3 | 0011 |
| 4 | 00101 |
| 5 | 011100 |
| 6 | 00100 |
| 7 | 0110 |
| 8 | 000 |
| 9 | 011101 |

- To store the string 0748901 we would get 000001110100100010010000001 using the fixed length code and 10110001010000111011010 using the variable length code.

# Huffman Tree: Optimal Binary Tree

- In order to be able to decode the variable length code properly it is necessary that it be a prefix code — that is, a code in which no codeword is a prefix of any other codeword.

- Decoding such a code is done using a binary tree, called Huffman tree.



# Huffman Coding

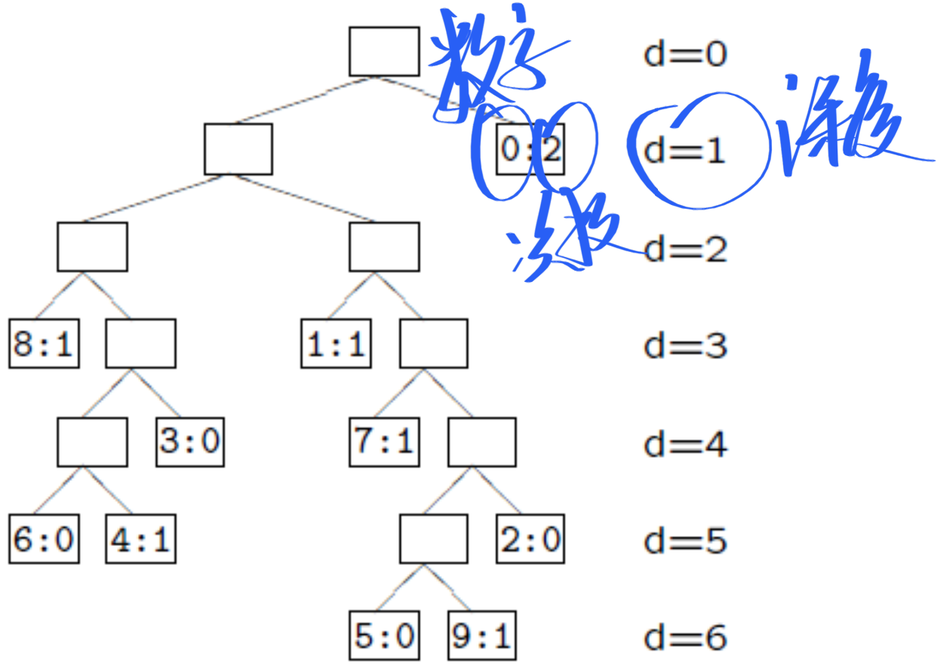
- Now assign to each leaf of the tree a value, $f(c)$, which is the frequency of occurrence of the character $c$ represented by the leaf.
- Let $d_T(c)$ be the depth of character $c$'s leaf in the tree $T$.
- Then the number of bits required to encode a file is

$$B(T) = \sum_{c \in C} f(c) d_T(c)$$

- which we define as the cost of the tree T.

# Huffman Coding

- For example, the number of bits required to store the string 0748901 can be computed from the tree *T*:




giving B(T) = 2×1+1×3+1×3+1×4+1×5+1×6 = 23.

Thus, the cost of the tree *T* is 23.

# Huffman Coding

- A tree representing an optimal code for a file is always a full binary tree (note, full v.s. complete, perfect) — namely, one where every node is either a leaf or has precisely two children.
- Therefore if we are dealing with an alphabet of $s$ symbols we can be sure that our tree has precisely $s$ leaves and $s-1$ internal nodes, each with two children.
- Huffman invented a greedy algorithm to construct such an optimal tree. The resulting code is called a Huffman code for that file.

# Huffman's Algorithm

- **Huffman's algorithm**: The algorithm starts by creating a forest of *s* single nodes, each representing one character, and each with an associated value, being the frequency of occurrence of that character. These values are placed into a priority queue (implemented as a linear array).

| | | | | |
|---|---|---|---|---|
| 5:1294 | 9:1525 | 6:2260 | 4:2561 | 2:4442 |
| 3:5960 | 7:6878 | 8:8865 | 1:11610 | 0:70784 |

- Then repeat the following procedure *s* − 1 times:
  - Remove from the priority queue the two nodes *L* and *R* with the lowest values, and create a internal node of the binary tree whose left child is *L* and right child *R*.
  - Compute the value of the new node as the sum of the values of *L* and *R* and insert this into the priority queue.

# Exercise

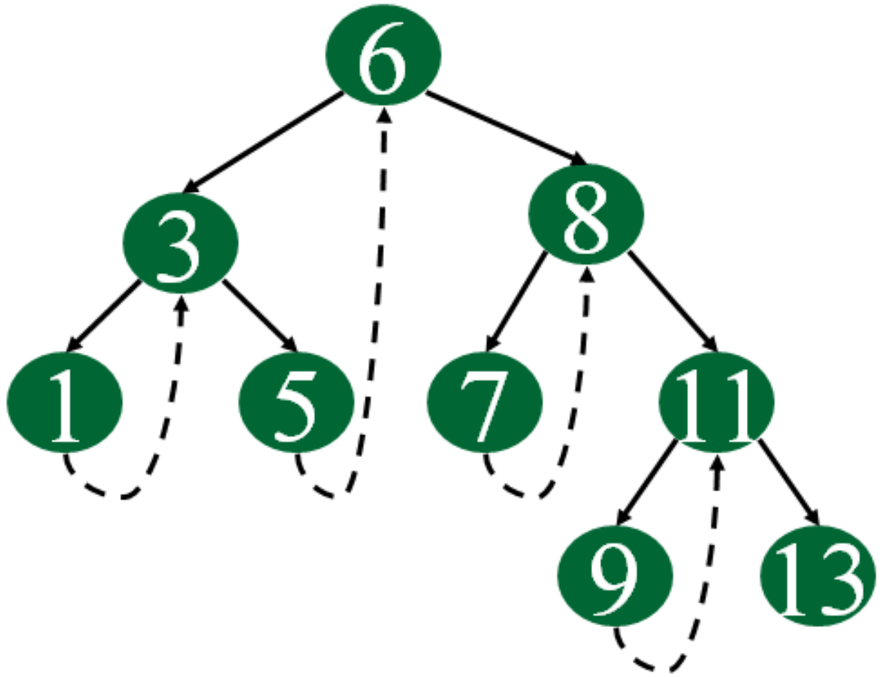
- Given Char = {a, b, c, d, e, f} and Freq = {8, 3, 4, 6, 5, 5}, build the Huffman Tree.

# Threaded Binary Trees

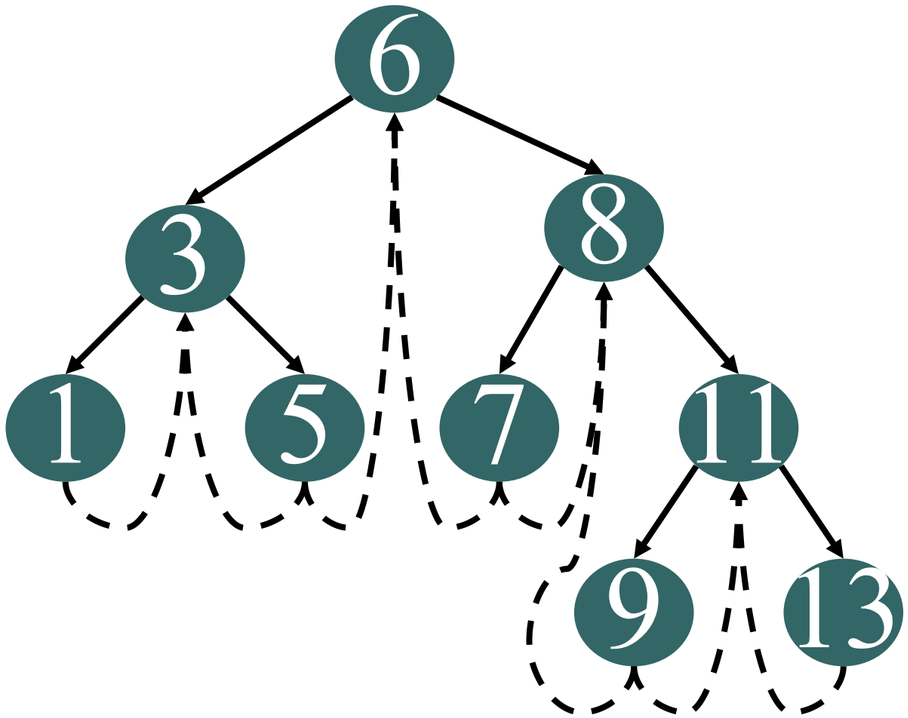
- A threaded binary tree is a type of binary tree data structure where the empty left and right child pointers in a binary tree are replaced with threads that link nodes directly to their in-order predecessor or successor, thereby providing a way to traverse the tree without using recursion or a stack.
- The idea of threaded binary trees is to make inorder traversal faster and do it without stack and without recursion. Threaded binary trees can be useful when space is a concern, as they can eliminate the need for a stack during traversal.
- There are two types of threaded binary trees.
  - ***Single Threaded:*** Where a NULL right pointers is made to point to the inorder successor (if successor exists)
  - ***Double Threaded:*** Where both left and right NULL pointers are made to point to inorder predecessor and inorder successor respectively. The predecessor threads are useful for reverse inorder traversal and postorder traversal.

# Threaded Binary Trees

- **Single Threaded**




- **Double Threaded**

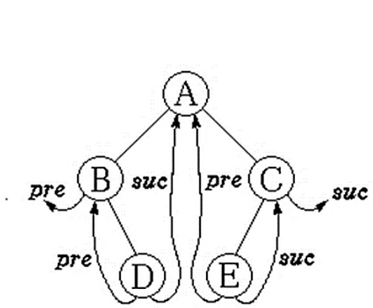


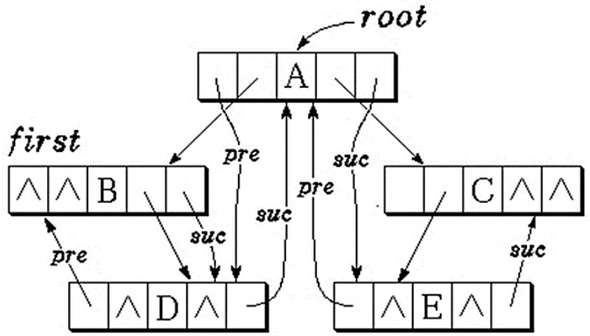

Inorder: 1 3 5 6 7 8 9 11 13

# Threaded Binary Trees

- Node definition







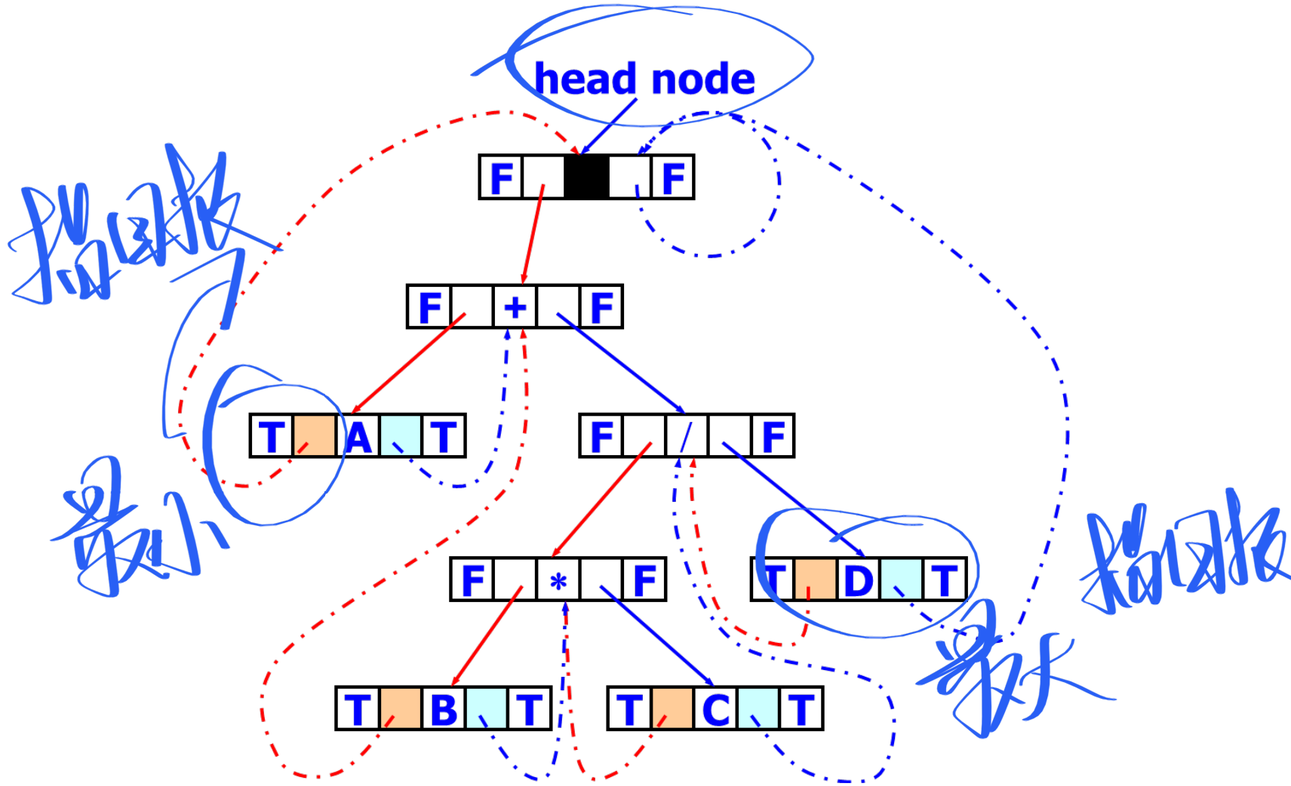
ltag=0: lchild points to left child

ltag=1: lchild points to predecessor

rtag=0: rchild points to right child

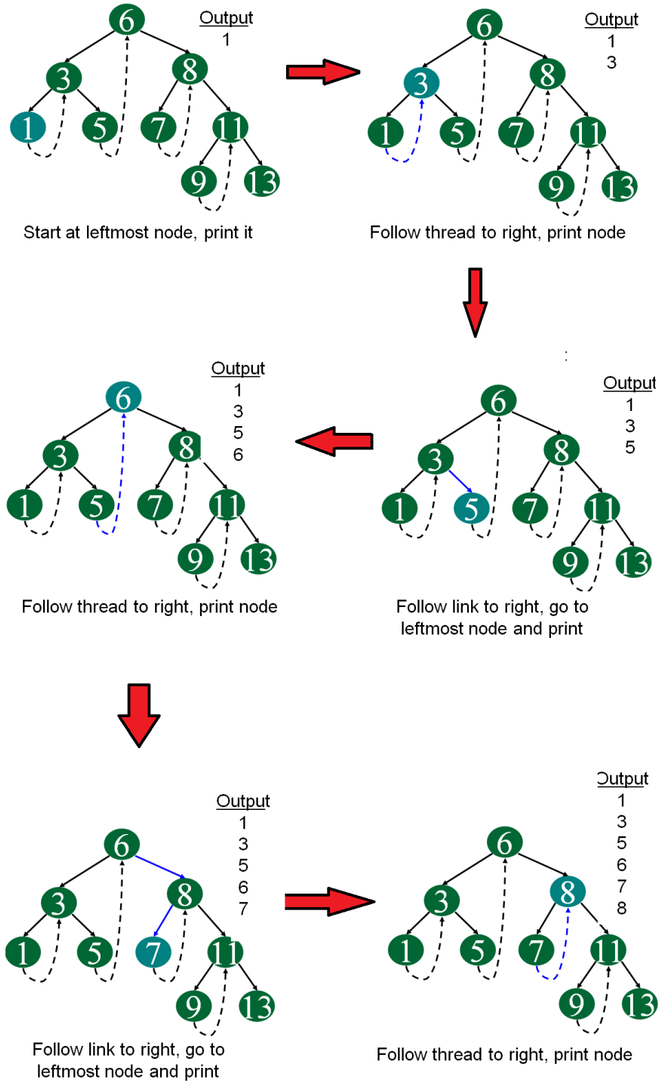
rtag=1: rchild points to successor

- Threaded binary tree with head node



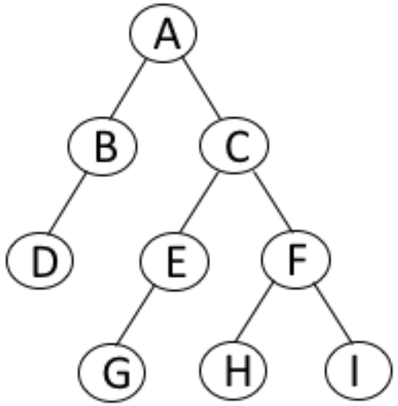
# Threaded Binary Tree Traversal

- Inorder traversal
  - We start at the leftmost node in the tree, print it, and follow its right thread.
  - If we follow a thread to the right, we output the node and continue to its right.
  - If we follow a link to the right, we go to the leftmost node, print it, and continue.

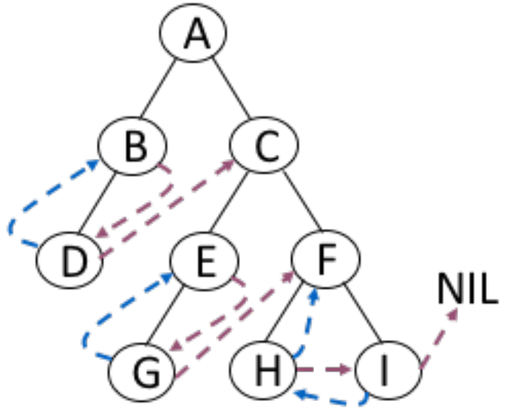



continue same way for remaining node.....

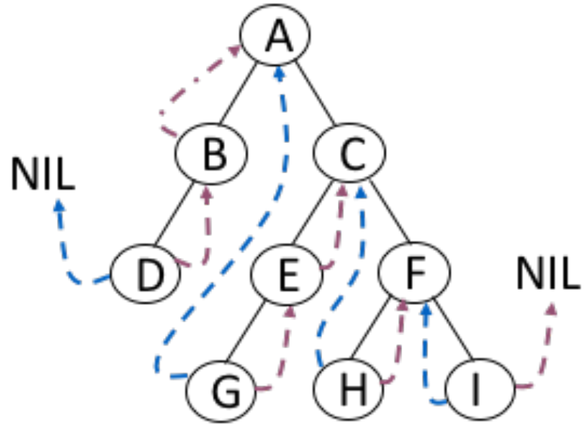
# Different Types of Threaded Binary Trees



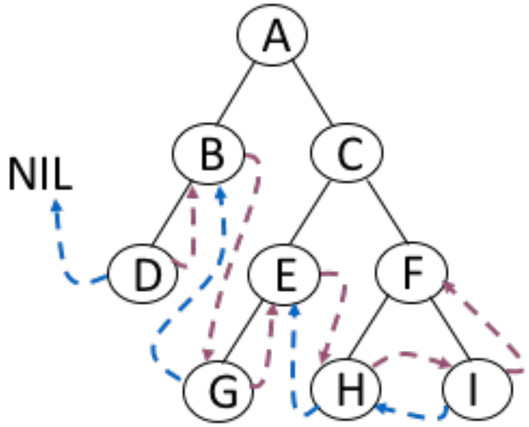

(a) 二叉树




(b) 先序线索树的逻辑形式
结点序列：ABDCEGFHI



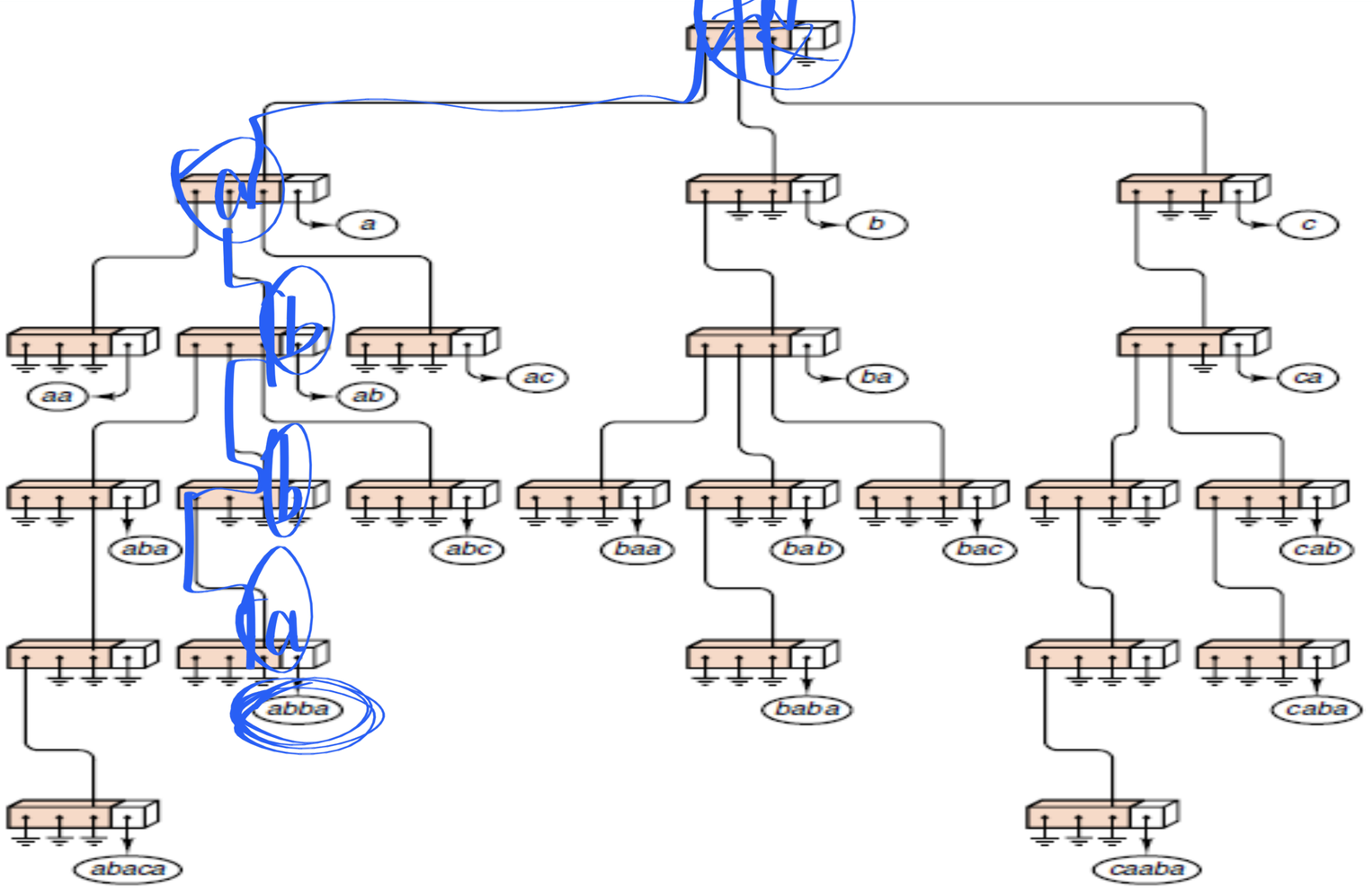

(c) 中序线索树的逻辑形式
结点序列：DBAGECHFI




(d) 后序线索树的逻辑形式
结点序列：DBGEHIFCA

# Lexicographic Search Trees: Tries*(字典树)

DEFINITION A ***trie*** of order $m$ is either empty or consists of an ordered sequence of exactly $m$ tries of order $m$.



# C++ Trie Declarations

- The term “trie” originated as letters extracted from the word *retrieval*, but it is usually pronounced like the word “try”.
- Every Record has a Key that is an alphanumeric string.

```
class Trie {
public:                          //    Add method prototypes here.

private:                         //    data members
   Trie_node *root;
};
const int num_chars = 28;
struct Trie_node {
//    data members
   Record *data;
   Trie_node *branch[num_chars];
//    constructors
   Trie_node();
};
```

**The constructor sets all pointer members to NULL!**

# Searching a Trie

```
Error_code Trie :: trie_search(const Key &target, Record &x) const
/* Post:  If the search is successful, a code of success is returned, and the output
          parameter x is set as a copy of the Trie's record that holds target. Otherwise,
          a code of not_present is returned.
   Uses:  Methods of class Key.  */

{
   int position = 0;
   char next_char;
   Trie_node *location = root;
   while (location != NULL &&
          (next_char = target.key_letter(position)) != ' ') {
      //   Terminate search for a NULL location or a blank in the target.
      location = location->branch[alphabetic_order(next_char)];
      //   Move down the appropriate branch of the trie.
      position++;
      //   Move to the next character of the target.
   }

   if (location != NULL && location->data != NULL) {
      x = *(location->data);
      return success;
   }
   else
      return not_present;
}
```

Determine which branch to move to

# Insertion into a Trie

```
Error_code Trie :: insert(const Record &new_entry)
/* Post:  If the Key of new_entry is already in the Trie, a code of duplicate_error
          is returned.  Otherwise, a code of success is returned and the Record
          new_entry is inserted into the Trie.
   Uses:  Methods of classes Record and Trie_node.  */
{
   Error_code result = success;
   if (root ==  NULL) root = new Trie_node;   //    Create a new empty Trie.
   int position = 0;                          //    indexes letters of new_entry
   char next_char;
   Trie_node *location = root;                //    moves through the Trie
   while (location  != NULL &&
          (next_char = new_entry.key_letter(position))  != ' ') {
      int next_position = alphabetic_order(next_char);
      if (location->branch[next_position] ==  NULL)
         location->branch[next_position] = new Trie_node;
      location = location->branch[next_position];
      position++;
   }
   //    At this point, we have tested for all nonblank characters of new_entry.
   if (location->data  != NULL) result = duplicate_error;
   else location->data = new Record(new_entry);
   return result;
}
```

Note the Trie_node constructor

# Trie

> The number of steps required to search a trie or insert into it is proportional to the number of characters making up a key, not to a logarithm of the number of keys as in other tree-based searches.

- A common application of a trie is storing a predictive text or autocomplete dictionary, such as found on a mobile telephone. Such applications take advantage of a trie's ability to quickly search for, insert, and delete entries.

# Splay Trees*: A Self-Adjusting Data Structure

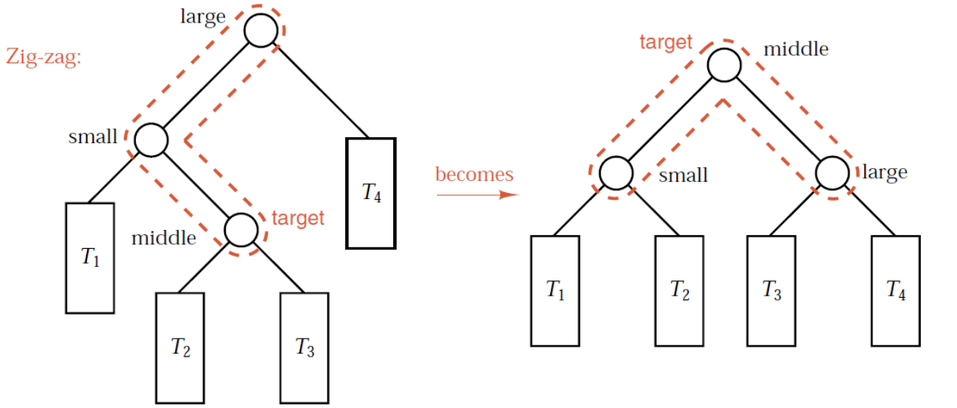
- In some applications, we wish to keep records that are newly inserted or frequently accessed very close to the root, while records that are inactive may be placed far off, near or in the leaves.
- We make a binary search tree into a *self-adjusting* data structure that *automatically* changes its shape to bring records closer to the root as they are more frequently accessed, allowing inactive records to drift slowly out toward the leaves.
- In a ***splay*** *tree*, every time we access a node, whether for insertion or retrieval, we lift the newly-accessed node all the way up to become the root of the modified tree.
- We use rotations like those of AVL trees, but now with *many* rotations done for *every* insertion or retrieval in the tree.

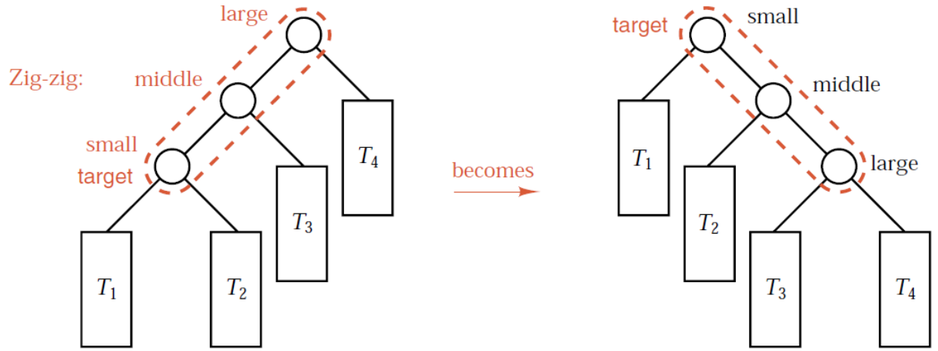
# Splay Rotations

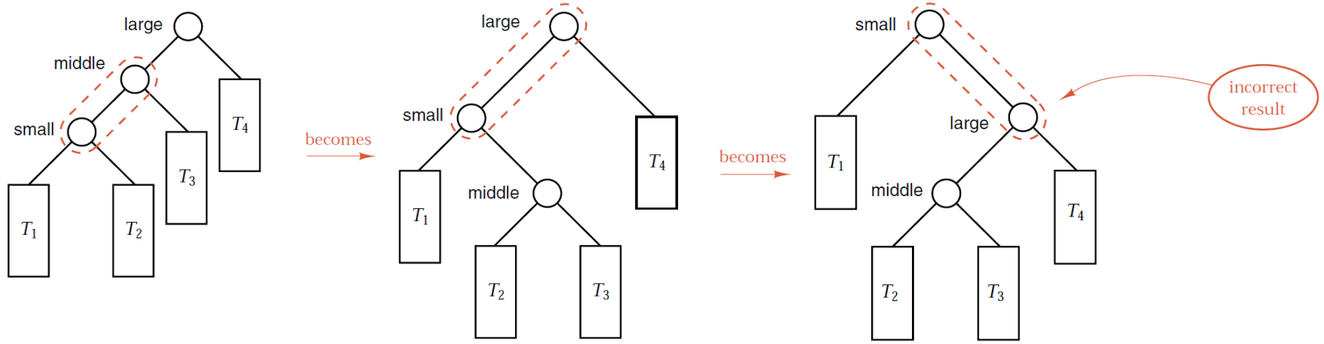
- We move the target node two levels up the tree at each step.
- Consider the path going from the root *down* to the accessed node.
  - If we move left, we say that we *zig*.
  - If we move right we say that we *zag*.
  - A move of two steps left (going down) is called *zig-zig*.
  - A move of two steps right is called *zag-zag.*
  - Left then right: *zig-zag*
  - Right then left: *zag-zig*.
- If the length of the path is odd, either a single *zig* move or a *zag* move occurs at the end.

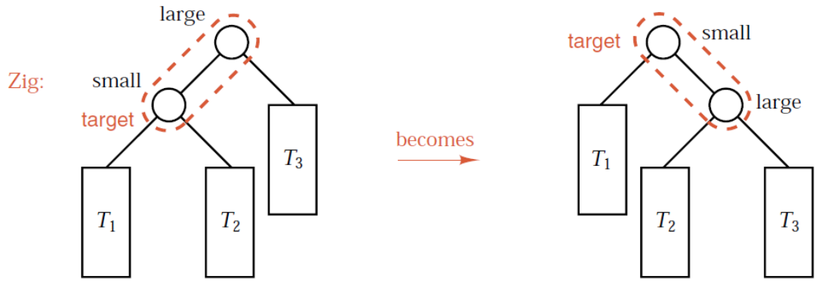
# Splay Rotations

- The zig-zag case is identical to that of an AVL double rotation.
- The zig case is identical to a single rotation.
- The zig-zig case is *not* the same as would be obtained by lifting the *target* node twice with single rotations (see bottom example).
  - It should be lifting the *middle* node first, then the *small* (target) node next
- Always think of lifting the target *two* levels at a time (except for a single *zig* or *zag* step at the end).

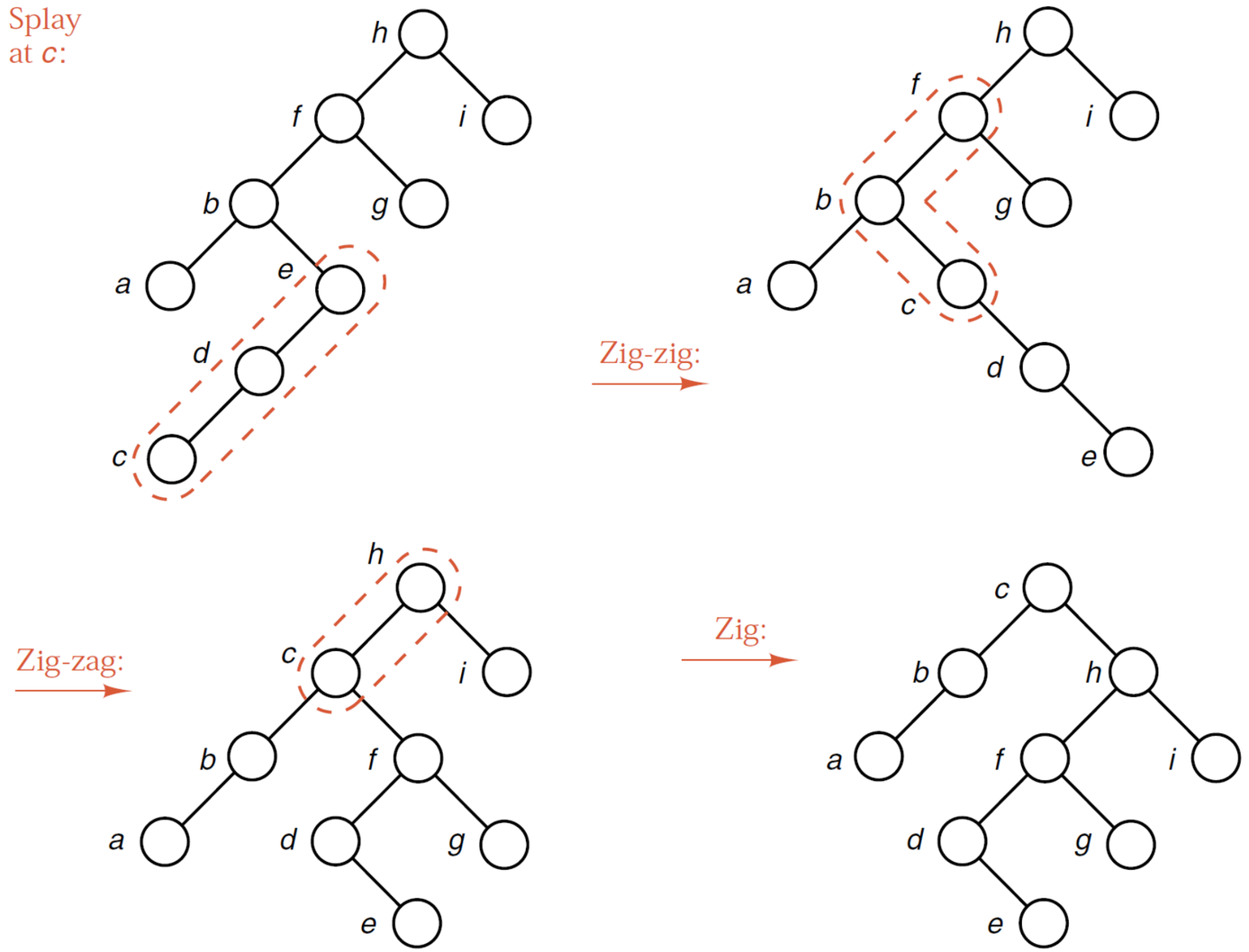


# Example


Splay
at c:
h
f
i
b
g
a
e
d
c
Zig-zig:
h
f
i
b
g
a
c
d
e
Zig-zag:
h
c
i
b
f
a
d
g
e
Zig:
c
b
h
a
f
i
d
g
e

# Top-Down and Bottom-Up Splaying

- By hand, we perform *bottom-up* splaying, beginning at the target node and moving up the path to the root two steps at a time. A single zig or zag move may occur at the top of the tree.
- Bottom-up splaying is essentially a two-pass method, first searching down the tree and then splaying the target up to the root.
- In a computer algorithm, we splay from the top-down while we are searching for the target node. When we find the target, it is immediately moved to the root of the tree, or, if the search is unsuccessful, a new root is created that holds the target. In top-down splaying, a single zig or zag move occurs at the bottom of the splaying process.
- If you run the splaying function we develop on the example trees, you will obtain the same results as doing bottom-up splaying by hand when the target is moved an even number of levels, but the results will be different when it moves an odd number of levels.

# Complexity of Splaying

**Theorem 10.10** The amortized cost of an insertion or retrieval with splaying in a binary search tree with $n$ nodes does not exceed

$$1 + 3 \lg n$$

upward moves of the target node in the tree.

- The fact that insertions and retrievals in a splay tree, over a long sequence, are *guaranteed* to take only *O(*log *n)* time.
- At any time, it is quite possible for a splay tree to degenerate into a highly unbalanced shape.

# Splay Tree

- https://www.bilibili.com/video/BV1py4y1a7v8

# Thank you!

